



PROTOCOL AUDIT REPORT

Relatio de Protocolo Auditu

Version 1.0

TACITVS

Auctor Princeps

August 6, 2025

AUDENTIA ET INTEGRITAS
Security Integrity Excellence

Prepared by: TACITVS Lead Security Researcher: - I.Leskov(TACITVS)

I. TABLE OF CONTENTS

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - The findings described in this document correspond the following commit hash:**
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Storing the password on-chain makes it visible to anyone, and no longer private
 - * [H-2] PasswordStore::setpassword has no access control, meaning a non-owner could change the password
 - Informational
 - * [I-1] The PasswordStore::getPassword natspec indicates a parameter that doesnt exist, causing the natspec to be incorrect.

II. PROTOCOL SUMMARY

Summarium Protocolli

PasswordStore is a protocol dedicated to storage and retrieval of a users passwords. The protocol is designed to be used by a single user, and not designed to be used by multiple users. Only the owner should be able to set and access this password.

III. DISCLAIMER

Cautela et Responsabilitas

The TACITVS team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

IV. RISK CLASSIFICATION

Classificatio Risci

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the [CodeHawks](#) severity matrix to determine severity. See the documentation for more details.

V. AUDIT DETAILS

Particularia Auditus

A. THE FINDINGS DESCRIBED IN THIS DOCUMENT CORRESPOND THE FOLLOWING COMMIT HASH:**

```
7d55682ddc4301a7b13ae9413095feffd9924566
```

B. SCOPE

Ambitus

```
./src/  
|-- PasswordStore.sol
```

C. ROLES

Personae

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read password.

VI. EXECUTIVE SUMMARY

Summarium Executivum

Add some notes about how the audit went, types of things you found, etc.

We spent X hours with Z auditors using Y tools, etc

A. ISSUES FOUND

Quaestiones Inventae

And get an output of

```
myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

Likelihood & Impact: - Impact: High - Likelihood: HIGH - Severity: HIGH

II. [H-2] PASSWORDSTORE::SETPASSWORD HAS NO ACCESS CONTROL, MEANING A NON-OWNER COULD CHANGE THE PASSWORD

Description: The PasswordStore::setpassword function is set to be an external function, however the natspec of the function and overall purpose of the smart contract is that the function allows only the owner to set a new password.

```
function setPassword(string memory newPassword) external {
-->    // @audit - There are no access controls
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact: Anyone can set/change the password of the contract, severely breaking the contract intended functionality.

Proof of Concept: Add the following to the PasswordStore.t.sol test file.

Code

```
function test_anyone_can_set_password(address randomAddress) public
{
    vm.assume(randomAddress != owner);
    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation: Add an access control conditional to the setPassword function.

```
if(msg.sender != s_owner){
    revert PasswordStore__NotOwner();
}
```

Likelihood & Impact: - Impact: High - Likelihood: HIGH - Severity: HIGH

A. INFORMATIONAL

Information

I. [I-1] THE `PASSWORDSTORE::GETPASSWORD` NATSPEC INDICATES A PARAMETER THAT DOESNT EXIST, CAUSING THE NATSPEC TO BE INCORRECT.

Description:

```
/*
 * @notice This allows only the owner to retrieve the password.
--> * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory) {
```

The `PasswordStore::getPassword` function signature is `getPassword()` which the natspec say it should be `getPassword(string)`

Impact: The natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```
- * @param newPassword The new password to set.
```

Likelihood & Impact: - Impact: NONE - Likelihood: HIGH? - Severity: Informational/Gas/Non-Crits